NP-hardness of Hadamard decomposition under a total rank budget

Working manuscript · Awaiting independent human expert review
15 September 2026

Abstract
We study the minimum sum of the ranks of two real matrices whose entrywise product is a prescribed rational matrix. We give a deterministic polynomial-time reduction from 3-SAT to deciding whether this sum is at most an input budget. The result holds for nonnegative integer input matrices and unrestricted-sign real factors. A square identity block forces every feasible allocation of the budget to have equal ranks. Additional pair-sum rows impose a partially specified Cartesian frame while retaining two-choice selectors. Calibration and a cycle of weighted rows encode subset product. The construction has a general converse for every admissible rank allocation. A deterministic decoder recovers a satisfying assignment from every valid decomposition encoded over one real algebraic primitive element. We give explicit polynomial bit bounds and separate the proof from finite computational checks.

1. Introduction
Hadamard decomposition represents a matrix as the entrywise product of low-rank matrices. Ciaperoni, Gionis and Mannila studied this representation for data analysis and left the exact real feasibility problem's complexity unresolved [1]. Subsequent numerical work explicitly compares decompositions under a total rank budget [2]. For a matrix with μ rows and N columns, factors of widths p, q use $(\mu + N)(p + q)$ scalar parameters in their dense parameterization. This makes the rank sum a natural representation cost.

For a rational matrix $M \in \mathbb{Q}^{\mu \times N}$, define

$$\mathcal{C}_0(M) = \min_{M = A \odot B} (\operatorname{rank}(A) + \operatorname{rank}(B)), \tag{1}$$

where $A, B \in \mathbb{R}^{\mu \times N}$ and $\odot$ denotes entrywise multiplication. There are exactly two factors. Their entries may have arbitrary signs, and rank zero is allowed; in particular, $\mathcal{C}_0(0) = 0$. The decision input is (M, b), where b is a nonnegative integer in binary. We ask whether $\mathcal{C}_0(M) \le b$.

A hardness result with prescribed bounds on the two ranks would not by itself settle this question. A matrix infeasible at one allocation could be feasible at another allocation with the same sum. Our construction excludes all such alternatives through equality in the rank-product bound.

Theorem 1 (Main result). Deciding $\mathcal{C}_0(M) \le b$ is NP-hard under deterministic polynomial-time many-one reductions, even for nonnegative integer matrices M. A 3-SAT formula with n occurring variables and m clauses maps to a matrix with a leading identity of order s^2 and budget

$$K = \max(3, 2n + 2m), \quad s = 2K, \quad b = 2s. \tag{2}$$

It is feasible exactly when the formula is satisfiable. Every feasible decomposition of the constructed matrix has both factor ranks equal to s. From every valid algebraically encoded decomposition, a deterministic algorithm recovers a satisfying assignment in time polynomial in the source length plus the supplied witness length.

The reduction uses the standard Karp framework [3]. Prime exponents encode 3-SAT as subset product. The matrix construction supplies the discrete choices: an incomplete Cartesian frame forces anchor directions while leaving two orientations at each selector. Weighted rows make the product of the selected integers equal a target. The frame, together with budget saturation, is the mechanism that addresses arbitrary rank allocations.

2. Budget saturation

A target witness can be written as

$$M = (UV) \odot (WZ), \tag{3}$$

where the four matrix dimensions are $\mu \times p$, $p \times N$, $\mu \times q$ and $q \times N$, with $p + q \leq b$. Expanding the ordinary products into rank-one terms gives the familiar inequality $\operatorname{rank}(M) \leq pq$. This inequality and identity examples are established ingredients of Hadamard decomposition [1], [2].

Lemma 1 (Unique budget allocation). Suppose M has s^2 columns and contains I_{s^2} as a row submatrix. Every decomposition satisfying Equation 3 with $p + q \leq 2s$ has $p = q = s$, and both ordinary products have rank exactly s.

Proof. The identity gives $\operatorname{rank}(M) = s^2$. Consequently,

$$s^2 \leq \operatorname{rank}(UV) \operatorname{rank}(WZ) \leq pq \leq \frac{(p+q)^2}{4} \leq s^2. \tag{4}$$

Equality throughout forces $p + q = 2s$, $p = q$, and both actual ranks to attain their respective widths. This also excludes zero widths and smaller budgets. □

For orientation, Figure 1 shows the leading block of an actual constructed YES instance. The extra rows used below constrain its possible factorizations; the identity alone does not impose a Cartesian basis.

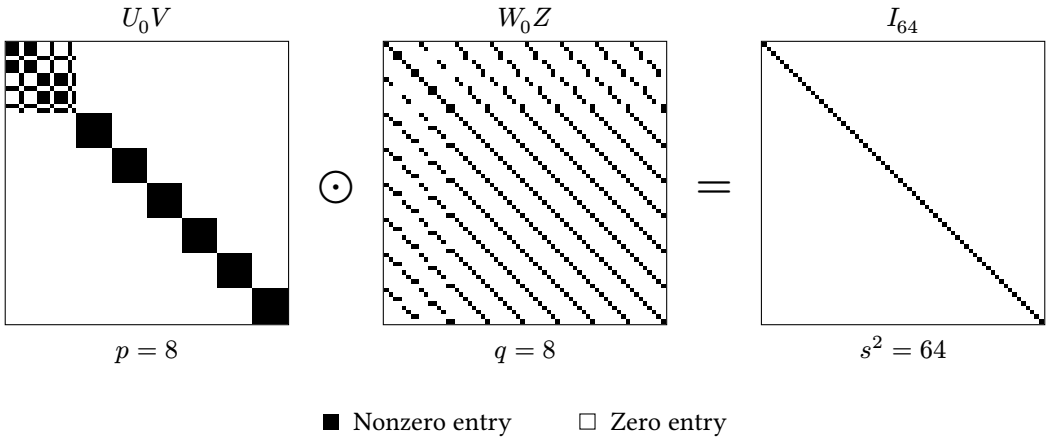


Figure 1: Support plots for the leading identity rows of an exact rational factorization constructed from $F = (x_1 \vee x_1 \vee x_1)$. Here $K = 4$, $s = 8$ and $b = 16$; U_0, W_0 denote the first 64 rows of U, W. Black cells mean nonzero entries, not unit values. All entries of the full 436×64 factorization were checked exactly. The displayed crop illustrates saturation; it does not display the additional constraint rows. The matrix presentation follows Figures 1 and 3 of [1].

3. Construction

3.1. Prime encoding

Let F have m clauses, each with three literals, allowing repetitions. Number its occurring variables consecutively from 1 to n; the empty conjunction has $n = 0$. Choose distinct primes π_v for variables and κ_j for clauses. For each variable v and truth value e, let $c_{v,e,j}$ count its literal occurrences in clause j made true by e. Create the two assignment items

$$r(v, e) = \pi_v \prod_{j=1}^{m} \kappa_j^{c_{v,e,j}}. \tag{5}$$

Append the two slack items κ_j, κ_j^2 for each clause, and set

$$T = \left(\prod_{v=1}^{n} \pi_v \right) \left(\prod_{j=1}^{m} \kappa_j^4 \right). \tag{6}$$

Assignment items are ordered by variable, true before false, followed by clause slack pairs. Pad with ones to obtain K items $r_0, \ldots, r_{K-1}$ as in Equation 2.

Lemma 2 (Subset product). A subset of the items has product T if and only if F is satisfiable. Its assignment-item choices recover a satisfying assignment.

Proof. The exponent of π_v forces exactly one of variable v's assignment items. For a clause with c true literal occurrences, its slack items can add exponent zero, one, two or three. Since $0 \le c \le 3$, exponent four is reachable exactly when $c \ge 1$. This proves both directions, including repeated literals. Padding by ones changes no constraint; the empty formula has $T = 1$. □

3.2. The incomplete anchor grid

Set $s = 2K$ and $N = s^2$. Index the columns of M by pairs (a, c) with $0 \le a, c < s$, in row-major order. Write e_J for the unit row at index J. For $0 \le i < K$, name four indices

$$L_i = (0, 2i), \quad R_i = (1, 2i+1), \quad X_i = (0, 2i+1), \quad Y_i = (1, 2i). \tag{7}$$

Call every index except the X_i, Y_i an anchor. These are labels for the construction; no coordinates of a future factorization are being assumed. The anchor pattern appears in Figure 2.

Begin M with I_N. For every pair of anchors in the same grid row, append their two-unit sum. Do the same for every pair in the same grid column. Order these rows first by grid row, then by grid column, using lexicographic pairs within each group. For each i, add four local pair rows:

$$e_{X_i} + e_{L_i}, \quad e_{X_i} + e_{R_i}, \quad e_{Y_i} + e_{L_i}, \quad e_{Y_i} + e_{R_i}. \tag{8}$$

Next append one calibration row per item,

$$e_{L_i} + e_{R_i} + e_{X_i} + e_{Y_i}. \tag{9}$$

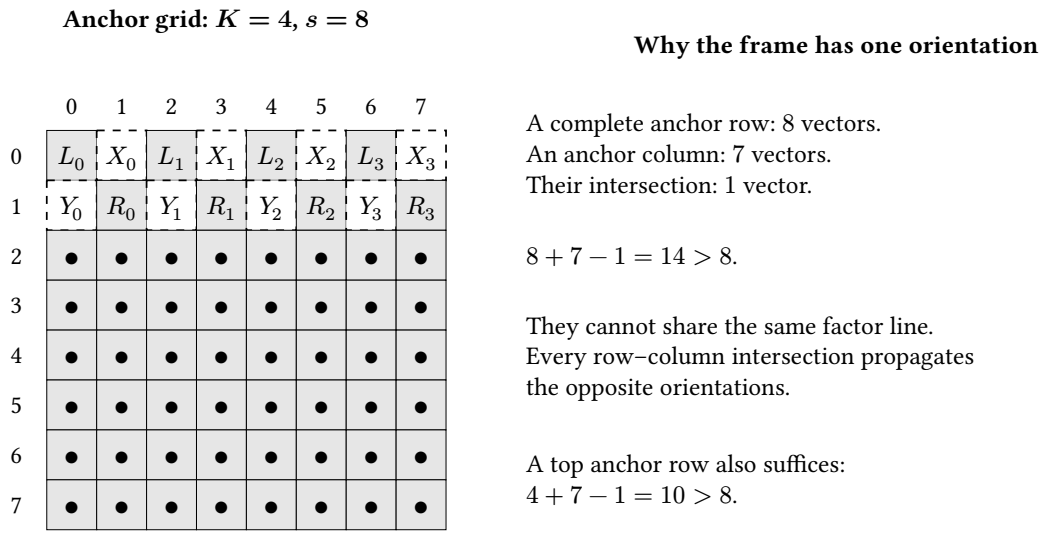


Figure 2: Construction indices for $K = 4$. Solid cells are anchors; dots denote the remaining anchor indices. Pair-sum rows join every pair of anchors within a grid row or column. Dashed cells X_i, Y_i have only the local pair constraints in Equation 8 before calibration and cycle constraints are added. The counts at the right explain why intersecting anchor cliques must use opposite factor sides. The grid is a schematic of index incidence, not a plot of matrix entries or a presupposed tensor coordinate system.

3.3. Weighted cycle rows

Finally add one cycle row for each item. Its nonzero coefficients in the ordered block (L_i, R_i, X_i, Y_i) are

$$(1, r_i, 1, r_i). \tag{10}$$

For $i < K-1$, its block $i+1$ is $(1,1,1,1)$. The final row instead uses block zero with coefficients

$$(1, T^2, T, T). \tag{11}$$

Every omitted entry is zero. The two blocks are distinct since $K \geq 3$. Set the output budget to $b = 2s$.

The identity, anchor pairs, local pairs, calibrations and cycles give

$$\mu = s^2 + K(K-1) + s(s-1)(s-2) + 6K \tag{12}$$

rows. The top two anchor rows contribute $K(K-1)$ pairs. The complete anchor rows and the anchor columns each contribute $s(s-1)\frac{s-2}{2}$ pairs. All entries of the resulting matrix are nonnegative integers.

4. Structure of every feasible decomposition

4.1. Product bases and pair sums

By Lemma 1, any feasible witness has widths (s, s). A nonzero tensor of the form $u \otimes w$ is a product vector; its coordinate matrix has rank one. Associate a row tensor $u_D \otimes w_D$ to every row D, and a column tensor $v_J \otimes \zeta_J$ to every column J. Their pairing equals $M_{D,J}$. The tensor interpretation is consistent with the rank-one reshape formulation in [4].

The leading identity pairs N row tensors with N column tensors as I_N. Since $\dim(\mathbb{R}^s \otimes \mathbb{R}^s) = N$, these families are dual bases. Let z_J denote the identity row tensor at index J. Every later row with coefficients m_J therefore has the uniquely determined tensor $\sum_J m_J z_J$.

Two independent product vectors have a product-vector sum exactly when they share a left or a right factor line. If neither line is shared, linear maps on the two independent pairs send their sum to a 2×2 identity, contradicting rank one. Sharing a line gives the converse by distributivity.

A clique of at least three independent product vectors with product-vector pair sums has one common factor line. Fix a pair sharing a left line and having distinct right lines. A third vector without that left line would have to share both distinct right lines. This is impossible, so the third vector shares the left line. The argument extends to the whole clique, and exchanging left and right gives the other case. Its common side is unique, since sharing both lines would make the vectors proportional.

4.2. Forcing the anchor frame

Lemma 3 (Anchor frame). After possibly exchanging the two factor spaces, there are bases $\alpha_0, \ldots, \alpha_{s-1}$ and $\beta_0, \ldots, \beta_{s-1}$ such that every anchor at index (a, c) is a nonzero scalar multiple of $\alpha_a \otimes \beta_c$. Each pair X_i, Y_i occupies the two opposite corners between the left lines α_0, α_1 and right lines β_{2i}, β_{2i+1}.

Proof. Every complete anchor row has s members, and every anchor column has $s-1$ members. Their pair constraints and the preceding clique argument give each a common side. A complete row and a column intersect once. If their sides agreed, their common lines would coincide at that intersection. Their union would place $2s-2 > s$ independent basis vectors on one factor line, in a subspace of dimension s. Thus their sides are opposite.

Every complete row intersects every column. All complete rows consequently have the same orientation, and all columns the opposite orientation. A top anchor row has K members and intersects each applicable column once. Equal orientation would confine $K + s - 2 > s$ independent vectors to dimension s, because $K \geq 3$. Both top rows therefore orient like the complete rows. After a possible global swap, write the anchor factors on the lines α_a and β_c as in the statement.

The s anchors in a complete row are independent, forcing the s right lines to be independent. In particular, each L_i, R_i pair has distinct right lines. Suppose the two top left lines coincided. A common neighbor X_i or Y_i without that left line would have to share both distinct right lines of its anchors. Hence every local neighbor would have the common top left line. All N basis vectors would then have left factors in the span of at most $s-1$ lines, contradicting their full span.

The top two left lines are therefore distinct. A common neighbor of L_i, R_i must occupy an off-diagonal corner between their two left and two right lines. The neighbors occupy different corners by independence. All identity tensors now use left factors from the s anchor-row lines. Their full span forces those lines to be independent as well. □

4.3. Calibrated selectors

Absorb the scalar of L_i into a vector b_i on line β_{2i}, and that of R_i into c_i on line β_{2i+1}. Put $\alpha = \alpha_0$ and $\beta = \alpha_1$. The anchor tensors are

$$z_{L_i} = \alpha \otimes b_i, \quad z_{R_i} = \beta \otimes c_i, \tag{13}$$

and all b_i, c_i together form a basis of the right factor space. The calibration row and Lemma 3 leave exactly two orientations:

$$\begin{aligned} \text{Selected:} \quad & z_{X_i} = t_i \alpha \otimes c_i, \quad z_{Y_i} = t_i^{-1} \beta \otimes b_i, \\ \text{Unselected:} \quad & z_{X_i} = t_i \beta \otimes b_i, \quad z_{Y_i} = t_i^{-1} \alpha \otimes c_i, \end{aligned} \tag{14}$$

with $t_i \neq 0$. Indeed, if the two initial multipliers are t_i, v_i, the calibration tensor has determinant $1 - t_i v_i$ in its local coordinates. Rank one forces $v_i = \frac{1}{t_i}$, without any sign assumption.

5. Cycle equivalence

Let ε_i be one for a selected item and zero otherwise. A coefficient block $(\xi_L, \xi_R, \xi_X, \xi_Y)$ at (L_i, R_i, X_i, Y_i) reshapes as

$$\begin{pmatrix} \xi_L & \xi_X t_i \\ \frac{\xi_Y}{t_i} & \xi_R \end{pmatrix} \ \text{(selected)}, \qquad \begin{pmatrix} \xi_L & \frac{\xi_Y}{t_i} \\ \xi_X t_i & \xi_R \end{pmatrix} \ \text{(unselected)}. \tag{15}$$

Define $h_i = \frac{1}{t_i}$ when selected and $h_i = t_i$ otherwise. The calibration, outgoing and closing patterns have zero determinant and nonzero entries. Their second-row to first-row ratios are respectively

$$\begin{aligned} (1,1,1,1) &\mapsto h_i, \\ (1, r_i, 1, r_i) &\mapsto r_i^{\varepsilon_i} h_i, \\ (1, T^2, T, T) &\mapsto T h_i. \end{aligned} \tag{16}$$

Lemma 4 (Cycle equivalence). The constructed matrix has total rank cost at most $2s$ if and only if some subset of its items has product T.

Proof. A feasible decomposition has the frame and selectors just proved. Independent right blocks make a two-block cycle tensor rank one exactly when its two left lines agree. The ratios in Equation 16 therefore give

$$h_{i+1} = r_i^{\varepsilon_i} h_i \quad (0 \le i < K-1), \quad T h_0 = r_{K-1}^{\varepsilon_{K-1}} h_{K-1}. \tag{17}$$

Since all h_i are nonzero, multiplication and cancellation yield

$$\prod_{i=0}^{K-1} r_i^{\varepsilon_i} = T. \tag{18}$$

Conversely, suppose Equation 18 holds. Choose standard bases for both factor spaces, put $h_0 = 1$, and compute successive prefix products using Equation 17. Use $t_i = \frac{1}{h_i}$ when selected and $t_i = h_i$ otherwise. Place every anchor at its standard grid cell and place X_i, Y_i at the corners and scales in Equation 14. These N vectors are a scaled permutation of the coordinate tensor basis. Their duals are consequently product vectors as well.

Anchor pair sums share a factor. Local pair sums do also. Calibration rows have zero local determinant, and cycle rows have equal left-line ratios, including the final row by Equation 18. Every prescribed row therefore has a rank-one reshape. Factoring each reshape supplies its two row vectors; factoring the dual basis supplies the column vectors. Their pairings reproduce every entry of M, including zeros, at widths (s, s). All these constructed factor coordinates are rational. □

Combining Lemmas 2 and 4 proves the decision equivalence in Theorem 1 for all rank allocations. The next section supplies its executable witness recovery.

6. Recovery and bit complexity

6.1. Decoding an arbitrary valid witness

The two identity rows L_0, L_1, with row-major indices $0, 2$, share a factor line on the oriented left side and have independent factors on the other side. Test whether rows 0 and 2 of U are proportional. If so, use U for decoding; otherwise use W. This determines the correct side even after a global factor exchange. Call the chosen matrix Q.

For each assignment item i, test whether row X_i, of index $2i + 1$, is proportional to row L_0 in Q. By Equation 14,

$$\varepsilon_i = 1 \Leftrightarrow Q_{X_i,*} \ \text{ is proportional to } \ Q_{L_0,*}. \tag{19}$$

Read the true item in each variable pair. Lemma 2 guarantees one selected assignment item per pair and satisfaction of every clause. The predicate depends on factor lines, so arbitrary invertible coordinate changes and nonzero gauges do not affect it.

6.2. Exact algebraic zero tests

The finite witness encoding uses one real algebraic primitive element θ. It supplies a nonzero dense integer polynomial P and rational non-root endpoints $a < b$ isolating exactly one distinct real root θ. Every factor coordinate is a dense rational polynomial evaluated at that root. The defining polynomial may be reducible or have repeated roots. Rational numerators and denominators are written as integer literals, without compressed exponents.

For a nonzero vector u, choose its first nonzero coordinate u_j. Another vector v is proportional precisely when

$$u_j v_l - v_j u_l = 0 \quad \text{for every coordinate } \ l. \tag{20}$$

Every identity-factor row is nonzero. To test a polynomial D at θ, use

$$D(\theta) = 0 \Leftrightarrow \gcd(P, D) \ \text{ has a root in } \ (a, b). \tag{21}$$

All roots of the gcd are roots of P, so the isolating premise makes its root count zero or one. This avoids factorization of P and construction of a new number field.

The executable takes the last nonzero subresultant for the gcd, clears rational denominators, removes multiplicities with the derivative gcd, and counts roots with a sign-corrected subresultant Sturm sequence. Endpoint variation counts omit zero evaluations. The required coefficient and algebraic-comparison bounds are standard [5]. The implementation calls SymPy's `Poly.subresultants` and `sturm_amv(method=1)` the latter uses controlled remainders and the Sturm signs discussed in Section 2.4.3 of [6]. No feasibility solver runs inside construction or recovery.

6.3. Polynomial bounds

Let ℓ be the source bit length and $S = n + m + 2$. The first $n + m$ primes have magnitude $O(S \log S)$ [7]. Trial division generates them in polynomial time. Counting all $3m$ literal occurrences gives $O(S \log S)$ bits per item, for T, and for T^2. The dimensions in Equation 2 and Equation 12 imply

$$N = O(S^2), \quad \mu = O(S^3), \quad \text{output bits} = O(S^6 \log S). \tag{22}$$

There are $O(S^5)$ dense entries. Schoolbook arithmetic and serialization fit the conservative bounds

$$O((\ell+2)^{10}) \text{ forward bit operations}, \quad O((\ell+2)^7) \text{ output bits.} \tag{23}$$

The rational completeness witness also has polynomial length: each h_i is an item subproduct, with $O(S^2 \log S)$ bits under a loose bound.

Let H be the total serialized witness length and set $D_0 = H + 2$. Explicit dense degrees and literal coefficients bound input degrees and heights by $O(D_0)$. Each determinant polynomial has degree $O(D_0)$ and height $O(D_0^2)$ after clearing denominators. Sylvester determinant bounds give $O(D_0^3)$ heights for normalized gcd data and $O(D_0^4)$ for the subsequent sign-corrected subresultants. Square-free division preserves polynomial height. These estimates use the controlled subresultant procedure actually called.

Conservatively allowing $O(D_0^3)$ rational coefficient operations per sequence and $O(D_0^8)$ bits per intermediate numerator or denominator gives $O(D_0^{24})$ bit operations per zero test under schoolbook arithmetic. This is a loose upper bound, not a performance estimate. Orientation detection, pivot searches and Equation 20 require $O((n+1)s)$ zero tests. Thus recovery takes

$$O((\ell+2)^2(H+2)^{24}) \text{ bit operations} \tag{24}$$

and outputs $O(\ell+1)$ bits. Reading the witness and repeated coordinate parsing are covered. No bound polynomial in source length alone is imposed on an arbitrary supplied witness.

The converse holds over all real factors. Executable recovery concerns the specified finite algebraic encoding. A nonempty real solution set of the rational factorization equations has a real-algebraic point, by real closed field transfer; this does not assert a polynomial certificate bound for arbitrary target inputs. Malformed source strings map outside the target language. Canonical renaming of occurring variables is polynomial-time. These observations complete Theorem 1.

7. Related work and scope

The total rank budget and extremal identity constructions are prior results: Theorem 1 and Appendices A–B of [2] study rank-product expressivity, including optimization over the number of factors. Our problem keeps exactly two factors and tests a prescribed matrix. Extremal identity realizations do not impose the extra anchor and cycle constraints needed in the reduction.

The rank-one reshape viewpoint is also prior work [4]. Hardness of matrix signing and entrywise powers [8] ties factor magnitudes or coordinates; dropping those restrictions does not supply a converse for independent Hadamard factors. Generic Hadamard–Hitchcock identifiability [9] addresses different hypotheses and does not decide exact membership for these specially constrained matrices.

The new construction combines the incomplete anchor frame with two-factor budget saturation. The selectors and cycle encode the source choices; the frame makes their converse valid under every allocation allowed by the total budget. Primary-literature searches through 15 September 2026 found no matching total-budget hardness theorem within the reviewed coverage. Unindexed manuscripts, exhaustive citation closure and overlooked implications of broader representation results remain outside that coverage.

The result concerns exact minimum rank sum. It establishes neither approximation hardness nor strong NP-hardness, hardness for a variable number of factors, or NP membership for unrestricted target inputs. Both ranks grow with the source. The nominal parameter count motivating Equation 1 does not measure minimal bit storage, statistical generalization, or typical training behavior. Independent human assessment of correctness and novelty remains necessary.

Bibliography

[1] M. Ciaperoni, A. Gionis, and H. Mannila, “The Hadamard decomposition problem,” *Data Mining and Knowledge Discovery*, vol. 38, pp. 2306–2347, 2024, doi: 10.1007/s10618-024-01033-y.

[2] S. Wertz, A. Vandaele, and N. Gillis, "Efficient algorithms for the Hadamard decomposition," *arXiv preprint arXiv:2504.13633*, 2025, [Online]. Available: https://arxiv.org/abs/2504.13633v2

[3] R. M. Karp, "Reducibility among Combinatorial Problems," in *Complexity of Computer Computations*, Plenum Press, 1972, pp. 85–103. doi: 10.1007/978-1-4684-2001-2_9.

[4] N. Gillis, S. Saha, S. Sicilia, and A. Vandaele, "Manifold-based Algorithms for the Hadamard Decomposition," *arXiv preprint arXiv:2605.28980*, 2026, [Online]. Available: https://arxiv.org/abs/2605.28980v1

[5] I. Z. Emiris, B. Mourrain, and E. P. Tsigaridas, "Real Algebraic Numbers: Complexity Analysis and Experimentation," in *Reliable Implementation of Real Number Algorithms: Theory and Practice*, in Lecture Notes in Computer Science 5045., Springer, 2008, pp. 57–82. doi: 10.1007/978-3-540-85521-7_4.

[6] A. Akritas, "Anna Johnson and Her Seminal Theorem of 1917," *Computer Tools in Education*, no. 2, pp. 13–35, 2016, [Online]. Available: https://faculty.e-ce.uth.gr/akritas/publications/100.pdf

[7] J. B. Rosser and L. Schoenfeld, "Approximate formulas for some functions of prime numbers," *Illinois Journal of Mathematics*, vol. 6, no. 1, pp. 64–94, 1962, doi: 10.1215/ijm/1255631807.

[8] N. Gillis, S. Saha, S. Sicilia, and A. Vandaele, "On the Complexity of Low-Rank Matrix Signing and Entrywise Power Matrix Factorization," *arXiv preprint arXiv:2607.04875*, 2026, [Online]. Available: https://arxiv.org/abs/2607.04875v2

[9] A. Oneto and N. Vannieuwenhoven, "Hadamard-Hitchcock decompositions: identifiability and computation," *arXiv preprint arXiv:2308.06597*, 2025, [Online]. Available: https://arxiv.org/abs/2308.06597v2

A. Verification and reproducibility

A.1. Evidence and limitations

The implementation uses Python 3.14.7 and SymPy 1.14.0. Prepared tests use Z3 5.1.0; this document uses Typst 0.15.1. The generic target oracle enumerates all unordered legal rank allocations and solves their defining quartic equations. Swapping factors makes this enumeration complete. An unknown outcome is an error, not a NO answer. Its hand-verifiable checks cover zero, rank-one and identity cases, algebraic witnesses, invalid products and budget violations.

The prepared suite injects six formulas and makes eight recovery calls, using both factor orders. A separately implemented verifier checks fourteen distinct formulas, twelve YES and two NO, and makes seventy recovery calls, including four algebraic gauges. Cases include all eight sign patterns of a one-variable clause, contradictory repeated unit clauses, and mixed cases with up to three variables and $K \leq 8$. The largest target has 3720 rows and 256 columns.

For these larger outputs, each checker validates the entire matrix against the frame family before solving its extracted subset-product coefficients. The prepared checker uses Boolean-controlled linear prefix equations in Z3. The other uses meet-in-the-middle enumeration and exhaustive source assignments. Both construct target factors independently of the candidate and check all matrix entries exactly before invoking recovery in a fresh process. Their structural NO conclusions depend on Lemmas 1–4; agreement of finite oracles does not prove those lemmas.

Witness variants include alternative subsets, coordinate changes mixing active and frame directions, signed row and column gauges, and entire-factor exchange. Algebraic tests use positive and negative quadratic roots, a cubic root and a repeated defining polynomial. A separate reviewer checked the general proof and built further full witnesses without importing candidate implementations. Those checks used negative prefix parameters, dense coordinate changes, an isolated fifth root with a reducible repeated defining polynomial, and degree-14 coordinate representatives, in both factor orders. The review requested no repair. Agent review and finite tests do not replace human expert verification.

A.2. Artifacts and commands

From the repository root, the current artifacts are under `campaigns/hadamard-total-budget/work/`. They include `algorithm.py`, `contract.md`, `proof.md`, `preparation.md`, `verification.md`, and this manuscript. Raw checks are under `rounds/001/` in the same campaign: `probe-repaired.txt`, `prepared-check-repaired.txt`, `independent-check.txt` and `figure-check.txt`. Independent review and its own checks are under `reviews/native/`. The selector-cycle mechanism and algebraic zero-test implementation originate in the preceding fixed-width campaign, `campaigns/hadamard-decomposition/` the allocation-forcing frame is the extension proved here. Earlier failures remain with their owning round. Run:

```
cd campaigns/hadamard-total-budget/work
python3 check.py --self-test
python3 check.py --oracle frame --candidate algorithm.py
python3 verify.py --candidate algorithm.py
python3 figures/build.py
typst compile manuscript.typ manuscript.pdf
```

All domains are finite and explicit. There are no configured solver, subprocess or campaign timeouts. The two executable maps are:

```
python3 algorithm.py < source.json > target.json
python3 algorithm.py --extract < request.json > assignment.json
```

The source has keys `n` and `clauses`. The target has `matrix` and `budget`. Recovery receives `source` and `target_solution` the latter includes `p`, `q`, the four factor matrices and `field`, using the common-primitive encoding in `contract.md`. Construction and recovery share no process state.